

BIG DATA ANALYTICS & APPLICATIONS

Lecture 01: The Evolution of Big Data

Aifee Aadil

What this lecture teaches:

1. How data grew so big (the data explosion).
2. Why normal databases (RDBMS) struggle with it.
3. What Big Data is, what types of data exist, and where it is used.

1. The Evolution of Data

Over the last 40 years, data changed in **3 big steps** (eras).

Companies → People → Machines

Era 1: Enterprise Automation (1980s–1990s)

Companies started using computers for daily business work.

Business work → Typed into computer → Stored in tables

Main systems used:

- **RDBMS** = Relational Database Management System → software that stores data in tables. (MySQL, Oracle, PostgreSQL)
- **ERP** = Enterprise Resource Planning → manages company work (money, stock, staff).
- **CRM** = Customer Relationship Management → keeps track of customers.
- **OLTP** = Online Transaction Processing → handles small daily transactions.

Example:

Customer buys a product → payment done → stock updated → order saved.
This whole thing is one OLTP transaction.

Data size: **Megabytes to Gigabytes**.

Simple:

Companies typed their business records into databases.

Era 2: Web 2.0 & User-Generated Content (2000s)

Web 2.0 = the web where **users create content**, not just read it.

People → Post / Upload / Search → Huge content

Example:

On Facebook, millions of people upload photos and posts every day.
The company did not create this data. The users did.

Came from: social networks, search engines, web portals, online shopping.

Data size: **Terabytes to Petabytes**.

Type: mostly **unstructured** (photos, videos, posts).

Simple:

People started creating the data.

Era 3: Machine-Generated & Industry 4.0 (2010s–Present)

Now **machines** create most of the data, non-stop, by themselves.

Sensors / Devices → Send data every second → Never stops

- **IoT** = Internet of Things → everyday devices connected to the internet (smart meters, smart watches, cars).
- **Industry 4.0** = smart factories using sensors + automation + data.
- **Autonomous systems** = machines that work by themselves (self-driving cars).

Example:

A smart meter sends your electricity usage every 15 minutes — no human types anything.

Data size: **Exabytes to Zettabytes**.

Type: **machine-to-machine continuous streams**.

Simple:

Machines create data all the time.

Easy Memory Trick:

Era 1 → **Companies** type data Era 2 → **People** post data Era 3 → **Machines** stream data

2. Units of Data

A **byte** = the smallest normal unit of data (about 1 letter).

Each next unit is **1000 times bigger**.

GB → TB → PB → EB → ZB → YB

Unit	Size	Real-life example
Gigabyte (GB)	10^9 bytes	1 hour of HD video (~3 GB)
Terabyte (TB)	$10^{12} = 1000$ GB	All printed books of the US Library of Congress (~20 TB)
Petabyte (PB)	$10^{15} = 1000$ TB	500 billion printed pages
Exabyte (EB)	$10^{18} = 1000$ PB	Whole internet traffic in 2004
Zettabyte (ZB)	$10^{21} = 1000$ EB	Whole world's digital data in 2016 (~16 ZB)
Yottabyte (YB)	$10^{24} = 1000$ ZB	Future world data-centre capacity

Easy Memory Trick:

Good Teachers Please Every Zealous Youth

G → T → P → E → Z → Y (each step ×1000)

3. The Data Explosion

How much data is there?

- World data has crossed **120 Zettabytes**.
- **Over 90%** of all data was created in the **last 3–5 years**.

Example:

1 ZB = 1 billion TB.

A normal laptop has 1 TB.

So 120 ZB ≈ **120 billion laptops** full of data!

Simple:

The data is far too big for any single computer.

Data grows faster than hardware

Moore's Law = computer chips become about 2× more powerful every couple of years.

But data grows **even faster** than that.

CPU fast → but data can't reach CPU fast enough → CPU waits → slow system

Bottleneck = the slowest part that slows everything down (like the narrow neck of a bottle).

The bottlenecks are:

- **Memory bandwidth** → how fast data moves between RAM and CPU.
- **Storage I/O speed** → how fast data is read from disk. (I/O = Input/Output)
- **Network latency** → delay when data travels over the network.

Simple:

The CPU is fast, but moving data to it is slow.

What is creating so much data?

1. Social Media & Online Shopping (every minute)

- Search engines → millions of searches.
- Video sites → hundreds of hours of video uploaded.
- Social media → hundreds of thousands of posts, photos, messages.
- Online shops → thousands of orders, clicks, cart adds — **every second**.

Example:

Daraz does not only save “Aadil bought a phone”.

It also saves: searched “phone” → viewed 12 phones → added 2 to cart → bought 1.

This is called **clickstream** = record of every click.

Simple:

Systems now record every action, not just purchases.

2. IoT & Sensors

- **Self-driving car** → 1–5 TB of data **per hour** (camera, radar, LIDAR).
- **Jet engine** → several TB per flight.
- **Smart meter** → reading every 15 minutes, in millions of homes.
- **Factory sensors** → thousands of readings per second (kHz).

Simple:

Sensors produce huge data automatically.

Data at Rest vs Data in Motion

1. Data at Rest (Old way)

Data is stored first. We ask questions sometimes.

Data → Store → Ask a query when needed

Example:

At the end of the month, the manager checks total sales.

Used in: Traditional databases (OLTP / OLAP)

OLAP = Online Analytical Processing → analysing old data for reports.

2. Data in Motion (New way)

Data keeps flowing. The query is **always** running.

Data → Process → Result (again and again, never stops)

Example:

A bank checks every card payment **the moment** it happens.

Sometimes data comes so fast that we process it **before** storing it.

Used in: Big Data streaming systems

Easy Memory Trick:

Rest → data **waits** Motion → data **flows**

4. Why RDBMS Struggles with Big Data

RDBMS was built on 3 assumptions:

1. Data fits neatly into **tables** (rows and columns).
2. Everything runs on **one powerful machine**.
3. Every change must follow strict **ACID** rules.

SQL = Structured Query Language → the language used to ask questions to a database.

What goes wrong?

Data becomes too big for one machine
↓
Data must be split across many machines
↓
JOINS and ACID locks now need network communication
↓
System becomes very slow

Problem 1: JOIN

JOIN = combining two tables using a common column.

Example:

Student table (ID, Name) + Course table (ID, Course) → JOIN → (Name, Course)

With billions of rows on different machines, JOIN becomes **very slow**.

Problem 2: ACID locking

Lock = while one person changes data, others must wait.

With many machines, all machines must **talk and agree** first → lots of waiting.

Simple:

RDBMS was made for one machine and neat tables. Big Data is neither.

5. Scale-Up vs Scale-Out

Scaling = increasing the system's power to handle more data.

1. Scale-Up (Vertical Scaling)

Make **one server stronger**.

Same Server → Add more CPU + RAM + Disk → Bigger Server

Example:

Your laptop is slow, so you upgrade RAM from 8 GB to 64 GB.

Problems:

- **Very expensive** — 2× power costs much more than 2× price.
- **Has a limit** — a machine can only hold so much RAM/CPU.
- **Single Point of Failure (SPOF)** — if that server dies, everything stops.

Simple:

Make one server bigger.

Used in: Traditional RDBMS

2. Scale-Out (Horizontal Scaling)

Add **more normal servers** and make them work together.

Server A + Server B + Server C + ... → Work Together

A group of servers working together = **Cluster**.

Cheap normal servers = **Commodity hardware**.

Example:

Instead of buying 1 super-expensive computer, buy 10 normal computers.

Advantages:

- **Fair cost** — 2 servers = about 2× cost, 2× power (linear).
- **Flexible** — add or remove servers without stopping the system.
- **Safe** — if one server fails, the others continue.

Simple:

Add more servers.

Used in: Modern Big Data platforms (Hadoop, Spark)

Easy Memory Trick:

Scale-UP → one **bigger** machine

Scale-OUT → **more** machines

6. Schema-on-Write vs Schema-on-Read

Schema = the structure or format of data.

1. Schema-on-Write

The schema is decided **before** storing data.

Data → Check Structure → Store

Example:

A database says every student must have:

ID, Name, Email

If data doesn't follow this format, it may be rejected.

- Writing is **slow** (checking takes time).
- Reading is **fast** (data is already clean).
- Hard to change the format later.
- Uses **ETL** = Extract → Transform → Load (clean first, then store).

Simple:

Decide the format first, then store the data.

Used in: Traditional RDBMS

2. Schema-on-Read

The data is stored first, and we decide its structure **when we need to use it**.

Data → Store First → Decide Structure When Reading

Example:

You can store student data with different information:

Student 1 → ID, Name

Student 2 → ID, Name, Email

Student 3 → ID, Name, Email, Phone

All can be stored without forcing the same structure first.

- Writing is **fast** (no checking).
- Reading needs **more work** (structure is decided then).

- Very easy to add new fields.

Simple:

Store first, decide the format later.

Used in: Big Data systems

Easy Memory Trick:

Schema-on-**Write**: Format → Store

Schema-on-**Read**: Store → Format

Which one to use?

- **Bank / exam results** → Schema-on-Write (wrong data must never enter).
- **Social media / sensor logs** → Schema-on-Read (formats change, data comes fast).

7. ACID vs BASE

Transaction = a group of steps that must be treated as one job.

Example: money transfer = take money from A + give money to B.

1. ACID (RDBMS world)

Strict rules so data is **always correct**.

A — Atomicity

All steps happen, or none happen.

Take 1000 Tk from A → Give 1000 Tk to B fails → Undo everything

Simple:

All or nothing.

C — Consistency

Data always follows the rules.

Example:

Rule: balance cannot be negative.

A withdrawal that makes balance −500 is refused.

Simple:

Data always stays valid.

I — Isolation

Many transactions at the same time do not disturb each other.

Example:

Two people try to buy the **last** concert ticket at the same time.
Only one gets it.

Simple:

Transactions do not mix.

D – Durability

Once saved (**committed**), data stays saved — even if power goes.

Simple:

Saved means saved forever.

Used in: Banking systems

Easy Memory Trick:

A → All or nothing **C** → Correct **I** → Independent **D** → Data stays saved

2. BASE (Big Data / NoSQL world)

NoSQL = “Not only SQL” → databases that don’t use strict tables (MongoDB, Cassandra).

Big Data keeps **copies** of data on many machines.

BASE accepts that copies may not match **for a short time**.

BA – Basically Available

The system keeps working even if some machines fail.

Example:

One Facebook server is down, but you can still scroll your feed.

S – Soft State

Data may change by itself, because copies are still updating in the background.

E – Eventually Consistent

Copies may differ now, but will **become the same soon**.

Example:

You change your profile picture.

Your friend may see the old picture for a few seconds.

Soon everyone sees the new one.

Update Server A → B, C still old → copy to B, C → all same

Simple:

Always available; correct a little later.

Used in: Social media feeds, click tracking

ACID vs BASE

	ACID	BASE
Focus	Always correct	Always available
Consistency	Immediately	Eventually
Used in	RDBMS, banking	NoSQL, social media

Neither is “better”. It depends on the job.

8. Storage: Shared Storage vs Shared-Nothing

1. Shared Storage (SAN / NAS)

Many servers use **one central storage**.

Server 1, 2, 3 ...100 → One Big Storage

- **SAN** = Storage Area Network → special network connecting servers to shared disks.
- **NAS** = Network Attached Storage → a storage box on the network shared by many computers.

Problem: when hundreds of servers ask for data at once, the storage connection gets **jammed**.

Example:

100 students, 1 photocopier machine → long queue.

Simple:

Everyone shares one storage → traffic jam.

2. Shared-Nothing

Every server has its **own** CPU, RAM, and disk.

Node A → CPU + RAM + Disk

Node B → CPU + RAM + Disk

Node C → CPU + RAM + Disk

They only talk through a normal network (**TCP/IP**).

Example:

100 students, 100 photocopier machines → no queue.

Add more nodes → speed grows equally (**linear**).

Simple:

Everyone has their own → no traffic jam.

Used in: Big Data clusters (Hadoop)

9. RDBMS vs Big Data — Full Comparison

Point	Traditional RDBMS	Big Data (Hadoop / Spark)
Data size	GB to TB	TB to EB
Data type	Structured tables only	Structured + semi + unstructured
Scaling	Scale-Up	Scale-Out
Schema	Schema-on-Write	Schema-on-Read
Consistency	ACID (immediate)	BASE (eventual)
Processing	Bring data to CPU	Send code to data
Hardware	Expensive special servers	Cheap normal servers
Language	SQL	MapReduce, Spark, Distributed SQL

10. Types of Data

Data is grouped by **how organised** it is.

Structured → Semi-Structured → Unstructured

(easy to process → harder to process)

Type	Share of world data
Structured	~10–15%
Semi-Structured	~15–20%
Unstructured	~70–80%

Most data (70–80%) is **unstructured** → that's why normal tables are not enough.

1. Structured Data

Data in a **fixed table** with fixed columns and types.

Rows + Columns + Fixed Rules

Example:

CustomerID	Name	Email	Balance
1001	Alice Smith	alice@example.com	4520.50
1002	Bob Jones	bob@example.org	120.00

- Types fixed: number, text, date.

- **Primary Key** = unique ID of each row.
- **Foreign Key** = links to a row in another table.

Simple:

Neat table. Every row has the same columns.

Used in: MySQL, Oracle, PostgreSQL, data warehouses. Queried with SQL.

2. Semi-Structured Data

No fixed table, but the data has **labels (keys/tags)** that explain itself.

Label : Value

Example:

(JSON)

```
{
  "transaction_id": "TXN-98421",
  "customer": { "id": 1001 },
  "items": [
    { "sku": "ITEM-A", "qty": 2, "price": 49.99 },
    { "sku": "ITEM-B", "qty": 1, "price": 12.50 }
  ]
}
```

"price": 49.99 → tells both the **name** and the **value**.

One order can have 2 items, another 50 → doesn't fit a fixed table.

Example:

(Web server log)

```
127.0.0.1 - frank [10/Oct/2025:13:55:36] "GET /image.gif" 200 2326
192.168.0.12 - - [10/Oct/2025:13:55:37] "POST /login.php" 401 1422
```

Has a pattern, but some fields are missing (-).

To read it we use **Regex** (text pattern matching) or tools like **Logstash**, **Fluentd**.

Formats: **JSON**, **XML**, **YAML**, **Avro**, **Parquet**.

Simple:

Has labels or a pattern, but not a fixed table.

3. Unstructured Data

No structure at all.

Example:

- Text → PDFs, emails, Word files, doctor's notes.

- Media → photos, videos, audio, X-ray / MRI scans.
- Signals → earthquake signals, radio readings.

Computers can't analyse it directly. It must be converted to numbers first:

Photo / Text / Audio → NLP or Computer Vision → Numbers → Analysis

- **NLP** = Natural Language Processing → computer understands human language.
- **Computer Vision** → computer understands images/video.

Simple:

No table, no labels. Must be converted before analysis.

Easy Memory Trick:

Structured → **Table** Semi → **Labels** Unstructured → **Nothing**

11. Data Warehouse vs Data Lake

1. Data Warehouse

Stores **clean, structured** data.

Extract → Transform (clean) → Load = ETL

Example:

A company stores clean monthly sales tables for manager reports.

- Clean first, then store.
- Expensive to keep lots of raw old data.
- Made for reports and **BI** (Business Intelligence = dashboards for managers).

Simple:

Organised library: everything cleaned and on the right shelf.

2. Data Lake

Stores **raw data of every type** as it is.

Extract → Load (store raw) → Transform later = ELT

Example:

Store all photos, logs, JSON, tables together; clean only when needed.

- Store first, clean later (Schema-on-Read).
- Cheap storage: **HDFS** (Hadoop Distributed File System), Amazon S3, Azure Blob.

- Made for **Machine Learning** and **Data Science**.

Simple:

Big storage hall: everything kept raw, organise later.

Easy Memory Trick:

Warehouse → **ETL** (clean then store) Lake → **ELT** (store then clean)

12. What is Big Data?

Wrong: “Big Data = a database bigger than 1 TB.” → **INCORRECT**.

Correct definition:

Big Data means the data, the tools, and the distributed systems we need when data is **too big**, **too fast**, or **too mixed** for a normal database on one machine.

Break it down:

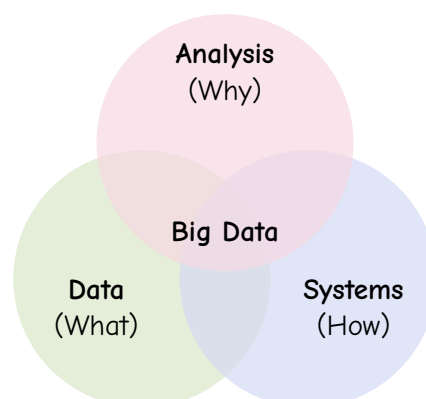
- **Too big** → size (volume).
- **Too fast** → speed of new data (velocity).
- **Too mixed** → many formats (heterogeneous = different kinds).
- **One machine** → single-node hardware.

Simple:

Big Data = when one normal database can't handle the data anymore.

The Big Data Triad

Big Data is **not one tool**. It is 3 things together:



1. **Data (What)** → huge, fast, mixed data from many sources.
2. **Computing Frameworks (How)** → many machines working together, fault-tolerant.
3. **Analytics (Why)** → machine learning, predictions, real-time analysis.

Easy Memory Trick:

What (data) + How (systems) + Why (analysis) = Big Data

The 6 Vs (preview)

1. Volume — how much

Data size (PB to EB).

Example: YouTube stores billions of videos.

2. Velocity — how fast

Speed of data: Batch → Micro-batch → Real-time.

Example: card payments every second.

3. Variety — how many types

Structured, semi-structured, unstructured.

Example: hospital has tables, X-rays, and doctor's notes.

4. Veracity — how trustworthy

Wrong, noisy, or missing data.

Example: broken GPS sends wrong location.

5. Valence — how connected

How much data items are linked to each other.

Example: Facebook: people linked to friends, pages, groups.

6. Value — how useful

Useful benefit we get from the data.

Example: using sales data to reduce waste.

Easy Memory Trick:

Size/Speed/Shape → Volume, Velocity, Variety

Trust/Links/Benefit → Veracity, Valence, Value

13. Distributed Computing Basics

Distributed computing = many computers share one big job.

Big Job → Split → Many Computers Work Together → Combine Results

Words to know:

- **Node** → one computer.
- **Cluster** → group of nodes.
- **Master** → the boss that gives out work.

- **Worker** → the node that does the work.
- **Parallel** → at the same time.

1. Data Locality

Send the **program to the data**, not the data to the program.

Moving Computation is Cheaper than Moving Data

Old way (move data)

10 TB file → travels over network → compute server

1 Gbps network $\approx$ 125 MB/sec.

10 TB $\div$ 125 MB/sec $\approx$ **22 hours** just to move it!

Big Data way (move code)

Small program (5 KB) → sent to Node 42 → runs on data already there

Data is kept in **blocks** (128 MB pieces).

The small program (a **JAR** file) goes to the node that has the block.

Example:

Instead of bringing 1000 students to one teacher,
send the teacher to the classroom where the students already are.

Simple:

Move the small program to the big data.

2. Partitioning

Cut a big file into **small pieces** and spread them over many machines.

1 TB file → 128 MB blocks → Node A, Node B, Node C ...

Example:

1 TB file → about 7,800 blocks of 128 MB → all nodes work at the same time.

Why?

One machine's RAM can't hold 1 TB. Splitting lets many machines work in parallel.

Simple:

Divide data into small parts.

3. Replication

Keep **copies** of each block on different machines.

Block 7 → Copy on Node A + Copy on Node F + Copy on Node K

- Default **Replication Factor** = 3 (3 copies).
- Copies are on **different racks** (rack = a cabinet of servers).

Why?

With 1000 cheap servers, **something fails every day**.

If one copy is lost, others are still there.

Cost: 1 TB data needs 3 TB disk space.

Simple:

Keep multiple copies.

4. Fault Tolerance (Self-Healing)

The system **keeps working** even when a machine fails.

Every node sends "I am alive" signal (**heartbeat**)



Node X crashes → heartbeat stops



Cluster manager notices



Makes a new copy of lost blocks on Node Y



Back to 3 copies — automatically

Simple:

Keep working even when something breaks.

Easy Memory Trick:

Partitioning → **Split** Replication → **Copy** Locality → **Go to data** Fault Tolerance → **Survive**

14. Batch vs Stream Processing

1. Batch Processing

Collect data first, then process it **all together**.

Collect → **Wait** → **Process Whole Batch** → **Result**

Example:

At midnight, calculate the whole day's sales.

- Data has an end (**bounded**).
- Handles huge data (**high throughput**).
- Slow result: minutes to hours (**high latency**).

Simple:

Collect first, process later.

Used in: MapReduce, Spark — monthly reports, training ML models

2. Stream Processing

Process each piece of data **as soon as it arrives**.

Data → **Process** → **Result** (continuously)

Example:

Your card is used in another country → bank blocks it in less than a second.

- Data never ends (**unbounded**).
- Result in under 1 second (**low latency**).
- One event at a time, or tiny groups (**micro-batch**).

Simple:

Process data as it arrives.

Used in: Flink, Spark Streaming — fraud detection, alerts

Latency = waiting time before you get the result.

Easy Memory Trick:

Batch → **Later**, big pile Stream → **Now**, one by one

15. Real-World Applications

1. Online Shopping — Personalization

Problem: millions of users shopping at once during sales.

Data: clicks, searches, mouse movement, past orders.

User clicks → **Kafka streams data** → **Recommendation in 50 ms**

- **Apache Kafka** → tool that carries live data streams.
- Recommendations update in **50 milliseconds**.
- Prices change based on stock and demand.

Simple:

Show each user the right product instantly.

2. Banking — Fraud Detection

Problem: thousands of card payments per second. Fraud must be stopped in **under 200 ms**. Old rule-based systems blocked many honest customers (**false positives**).

Batch: learn from old data → **Stream:** check each new payment live

- **Batch** → petabytes of old transactions train a model to spot unusual behaviour.
- **Stream** → each new payment is checked against location, shop type, and device.

Simple:

Learn from the past, check every payment live.

3. Factories — Predictive Maintenance

Problem: a machine breakdown costs **millions of dollars per hour**.

Sensors (100 readings/sec) → Spot tiny wear → Fix weeks before failure

Results: **40% fewer** breakdowns, longer machine life, fewer spare parts needed.

Simple:

Fix the machine before it breaks.

4. Smart Cities — Traffic

Data: bus GPS, phone locations, road sensors, traffic cameras.
(Video = unstructured, GPS = semi-structured.)

- Traffic lights change timing based on live traffic.
- Green path cleared for ambulances.
- More buses sent when crowds are expected.

Simple:

Use live city data to reduce traffic.

5. Healthcare — Genomics

- One person's DNA (genome) = **100–200 GB** of data.
- Millions of patients → **petabytes** in a data lake.
- **GWAS** (Genome-Wide Association Studies) → link genes to how people react to medicine.
- Track disease outbreaks and virus mutations in real time.

Simple:

Study huge DNA data to find better treatments.

16. Quick Revision

Remember

- Data eras: Companies → People → Machines.
- World data > 120 ZB; data grows faster than hardware.
- RDBMS: one machine, tables, ACID → fails at Big Data scale.
- Scale-Up = bigger machine. Scale-Out = more machines.
- Schema-on-Write = format → store. Schema-on-Read = store → format.
- ACID = always correct. BASE = always available, correct later.
- Shared storage = queue. Shared-nothing = each has its own.
- Structured (table), Semi (labels), Unstructured (nothing) — 70–80% is unstructured.
- Warehouse = ETL, clean data. Lake = ELT, raw data.
- Locality = move code to data. Partition = split. Replication = 3 copies.
- Batch = later, big pile. Stream = now, one by one.

17. Exam Questions

Q1. Why is adding $2\times$ RAM to one server (Scale-Up) much costlier than buying 2 normal servers (Scale-Out)?

Ans: Very high-end parts are rare and special, so price rises much faster than power. One machine also has a physical limit. Normal servers are cheap and mass-produced, so 2 servers $\approx 2\times$ cost for $2\times$ power. Scale-Out also has no single point of failure.

Q2. Give one system that needs Schema-on-Write and one that prefers Schema-on-Read.

Ans: Banking needs Schema-on-Write — wrong data must be rejected. Social media / sensor logs prefer Schema-on-Read — data comes fast in changing formats.

Q3. Why is sending code to data better than bringing data to code for 50 TB?

Ans: Moving 50 TB on a 1 Gbps network takes about 4–5 days. The program is only a few KB. Sending the program to each node that already has the data lets all nodes work at the same time, using their own disk and RAM. Only small code and results travel on the network.

Practice Questions

1. Explain the 3 eras of data.
2. Define Big Data. Why is “bigger than 1 TB” wrong?
3. Why does RDBMS struggle with Big Data?
4. Compare Scale-Up and Scale-Out.
5. Explain ACID and BASE with examples.
6. Compare shared storage (SAN/NAS) and shared-nothing.
7. Explain structured, semi-structured, and unstructured data with examples.

8. Compare Data Warehouse and Data Lake (ETL vs ELT).
9. What is the Big Data Triad?
10. Write the 6 Vs with examples.
11. Explain partitioning, replication, and fault tolerance.
12. Compare batch and stream processing.
13. How does Big Data help detect card fraud?